

Linear and logistic regression

A focused guide to the formulas used in applications A, B, and C

1. Linear regression: predict a numerical quantity

Let x_i describe an observation and y_i its measured outcome. Linear regression predicts a conditional mean: the average outcome at those features. For one feature,

$$\hat{y}_i = b + wx_i, \quad r_i = y_i - \hat{y}_i.$$

The intercept b is the prediction at $x = 0$; the slope w is the predicted change in y for one extra unit of x . A latency slope of 8 ms/MB means one more MB corresponds to 8 additional predicted milliseconds. This is a description of the fitted relationship, not automatically a causal effect.

Choose the line by least squares. Minimize the residual sum of squares:

$$\text{RSS}(b, w) = \sum_{i=1}^n (y_i - b - wx_i)^2.$$

Squaring prevents positive and negative errors from canceling and emphasizes large misses. At the minimum, differentiating with respect to b gives $\sum_i r_i = 0$, hence $b = \bar{y} - w\bar{x}$. Substitution and differentiation with respect to w give

$$\hat{w} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \quad \hat{b} = \bar{y} - \hat{w}\bar{x}.$$

The denominator must be positive: a constant feature cannot identify a slope. The numerator measures how x and y move together; the denominator accounts for the scale of x .

A calculation to keep in mind.

x	0	1	2	3
y	1	2	2	4
$\hat{y} = 0.9 + 0.9x$	0.9	1.8	2.7	3.6
$r = y - \hat{y}$	0.1	0.2	-0.7	0.4

Here $\bar{x} = 1.5$, $\bar{y} = 2.25$, the numerator is 4.5, the denominator is 5, and RSS is 0.70. The constant prediction $\bar{y}$ has RSS 4.75. A better fit on these four rows does not establish better predictions on new rows.

Measure prediction error on held-out observations. For m evaluation rows,

$$\text{MAE} = \frac{1}{m} \sum_i |y_i - \hat{y}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{m} \sum_i (y_i - \hat{y}_i)^2}.$$

Both have the units of y . RMSE penalizes large misses more strongly. Always compare with a constant baseline fitted as the *training* mean.

Remember. Least squares fits an average relationship. Individual outcomes still vary, and outliers can move the fitted line.

2. Multiple features and a readable optimizer

For request payload s and queue length q , use

$$\hat{y} = b + w_s s + w_q q.$$

The slope w_s describes a one-unit increase in payload *holding queue length fixed*. Sample coefficients need not equal the simulator's coefficients: the sample includes noise and only finitely many observations.

Put the intercept in the matrix. Each design row is $X_i = (1, x_{i1}, \dots, x_{ip})$, so

$$X \in \mathbb{R}^{n \times (p+1)}, \quad \beta = (b, w_1, \dots, w_p)^\top, \quad \hat{y} = X\beta.$$

Use one-dimensional arrays of length n for targets and length $p+1$ for coefficients. Accidental shapes such as $(n, 1)$ versus $(n,)$ can produce unintended broadcasting in array code.

Fit by a linear algebra solver. The convenient normalized objective and gradient are

$$J(\beta) = \frac{1}{2n} \|X\beta - y\|^2, \quad \nabla J(\beta) = \frac{1}{n} X^\top (X\beta - y).$$

The factor $1/2$ cancels the factor 2 from differentiation; it does not change the fitted least-squares solution. Setting the gradient to zero gives $X^\top X \hat{\beta} = X^\top y$. In code, use `np.linalg.lstsq(X, y, rcond=None)` rather than explicitly inverting $X^\top X$. A least-squares solver also handles rank deficiency.

Fit by gradient descent to understand learning.

$$\beta \leftarrow \beta - \eta \nabla J(\beta).$$

Compute predictions, subtract observations, average their feature-weighted errors, and take a small step in the opposite direction. Stop when the gradient norm is small, with a maximum iteration count. Plot the objective and report whether the stopping criterion was reached.

For this quadratic objective, a safe step is $\eta = 1/L$ with $L = \|X\|_2^2/n$, where $\|X\|_2$ is the largest singular value. The notebook computes this from the small design matrix instead of guessing a learning rate. Standardizing features usually improves conditioning and speed; agreement with the direct solver checks the implementation.

Collinearity: coefficients can be less stable than predictions. If $s' = s$, then $w_s s + w_{s'} s' = (w_s + w_{s'}) s$: only the sum is identified. If s' is merely close to s , small data changes can move the two slopes in opposite directions. Predictions near the observed feature relationship may remain stable. Predictions away from that relationship need not.

Residual plots. Plot training residuals against fitted values and individual features. Curves suggest missing structure; changing spread suggests nonconstant noise. A visually acceptable residual plot does not establish causality or guarantee out-of-sample accuracy.

Remember. Compare custom and reference solvers on the same objective and data. Compare model quality on separate observations. These are different checks.

3. Scaling, interactions, and controlling complexity

Learn feature scales from training rows. For a feature with training mean μ_j and positive scale a_j , use

$$z_j = (x_j - \mu_j)/a_j.$$

Reuse those values on validation and test rows. A coefficient on z_j measures a change per one scale unit, not per one original unit. Convert a scaled fit back with

$$w_j^{\text{original}} = w_j^{\text{scaled}}/a_j, \quad b^{\text{original}} = b^{\text{scaled}} - \sum_j w_j^{\text{original}} \mu_j.$$

Unpenalized regression with an intercept has the same prediction family under invertible scaling. Penalized fits depend on the scales, so standardization is part of the model specification. Handle constant features explicitly.

Ridge stabilizes slopes by shrinking them.

$$J_\lambda(\beta) = \frac{1}{2n} \|X\beta - y\|^2 + \frac{\lambda}{2} \sum_{j=1}^p w_j^2.$$

The intercept is not penalized. Larger λ places more weight on small slopes. This can reduce variability at the cost of bias; improvement is checked, not presumed. Ridge usually shrinks rather than deletes features. Its gradient adds $\lambda(0, w_1, \dots, w_p)^\top$. The notebook's reference `Ridge` uses `RSS` plus `alpha` times the squared slope norm, so use `alpha = nλ`, with the current fitting sample size, including inside a CV fold.

An interaction makes one slope depend on another feature. For advertising budgets in thousands of dollars and sales in thousands of units,

$$\widehat{\text{sales}} = b + w_T \text{TV} + w_R \text{radio} + w_{TR} \text{TV radio}.$$

An extra \$1,000 of TV spending corresponds to a predicted sales change $w_T + w_{TR}r$ thousand units at radio budget r . At $r = 30$, use $1000(w_T + 30w_{TR})$ to express the change in units. Retain both main effects when adding this interaction. The model remains linear in its coefficients.

A small, honest selection protocol.

1. Reserve 60% training, 20% validation, and 20% test before exploring target relationships. Use stratification for binary labels.
2. Select candidates or penalties using five-fold CV on training rows only. Refit preprocessing inside each fold; average its held-out scores.
3. Refit the selected model on the training partition. Use validation for an audit or a decision threshold. Freeze the model and policy.
4. Evaluate the frozen choices on the final test. Do not revise them from the test results and still call that test independent.

For these independent examples, random splits are appropriate teaching choices. Repeated customers or future-time prediction would require different splits.

Remember. A coefficient describes a conditional association. Adding a useful predictor or interaction does not turn observational data into an experiment.

4. Logistic regression: predict an event probability

For a binary target $y \in \{0, 1\}$, turn a linear score into a probability:

$$z = b + x^\top w, \quad \pi(x) = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

The score can be any real number, while $0 < \pi < 1$. At scores $-2, 0, 2$, the probabilities are approximately 0.119, 0.5, 0.881. Logistic regression estimates probabilities; a decision threshold is a separate choice made after fitting.

Coefficients change log odds.

$$\log \frac{\pi(x)}{1 - \pi(x)} = b + x^\top w.$$

Increasing feature j by one original unit multiplies the odds by e^{w_j} , holding other features fixed. It does not multiply the probability by this amount. The probability change depends on the starting score. For example, $z = -2 + 0.7x_1 - 0.4x_2$ at $(2, 1)$ gives $z = -1$ and $\pi \approx 0.269$.

Fit probabilities using log loss. A Bernoulli observation has likelihood $\pi_i^{y_i}(1 - \pi_i)^{1-y_i}$. Taking the negative logarithm and averaging gives

$$J(\beta) = -\frac{1}{n} \sum_i [y_i \log \pi_i + (1 - y_i) \log(1 - \pi_i)].$$

A confident wrong prediction is expensive: predicting 0.01 for an event that occurs contributes $-\log(0.01) \approx 4.605$. Predicting 0.9 contributes only 0.105. Optimize probabilities, not thresholded classification accuracy.

The gradient has the same pattern as linear regression. With the intercept included in X and $\pi = \sigma(X\beta)$,

$$\nabla J(\beta) = \frac{1}{n} X^\top (\pi - y).$$

For one row $(1, 2)$ with $y = 1$, zero coefficients give $\pi = 0.5$ and gradient $(-0.5, -1)$. A step of size 0.1 gives coefficients $(0.05, 0.1)$, score 0.25, and probability approximately 0.562. The update moves toward the observed label.

Use numerically stable code. Compute probabilities with `scipy.special.expit`. For binary labels, each log-loss term is

$$\log(1 + e^{(1-2y_i)z_i}),$$

evaluated as `np.logaddexp(0, (1 - 2*y) * scores)`. This avoids overflowing exponentials and taking a logarithm of a rounded zero probability. Check the analytic gradient against central finite differences before optimizing.

Regularize when needed. Add $\lambda \|w\|^2/2$ to mean log loss, leaving the intercept free. The gradient adds $\lambda(0, w^\top)^\top$. With perfectly separable labels, an unpenalized logistic fit can drive slopes toward infinity instead of having a finite optimum. A positive slope penalty controls this behavior when both classes are present.

5. Probability quality and decision quality

Evaluate probabilities before choosing a threshold. Compare mean log loss and the Brier score,

$$\text{Brier} = \frac{1}{m} \sum_i (y_i - \hat{\pi}_i)^2,$$

with a baseline predicting the *training prevalence* $\bar{y}_{\text{train}}$ for every row. Smaller values are better. A calibration plot groups predictions and compares each group's mean probability with its observed event rate. Small groups make this plot noisy.

Make a decision using a threshold t . Predict event 1 when $\hat{\pi} \geq t$. Lowering t typically catches more events but also raises more false alarms. In the confusion matrix, rows are actual labels and columns are predicted labels:

$$\begin{pmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{pmatrix}.$$

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

Precision asks how often an alarm is correct; recall asks how many events are caught. State a convention if a denominator is zero. A precision–recall curve shows their tradeoff over thresholds. Accuracy alone can conceal missed events when the event is rare.

Costs give a useful theoretical threshold. Suppose a false positive costs c_{FP} , a false negative costs c_{FN} , and correct decisions cost zero. At event probability π ,

$$\text{cost of predicting 1} = c_{\text{FP}}(1 - \pi), \quad \text{cost of predicting 0} = c_{\text{FN}}\pi.$$

Choose 1 when its expected cost is no larger, giving

$$t = \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}}.$$

For costs 1 and 5, $t = 1/6$. At probability 0.2, expected costs are 0.8 for an alarm and 1.0 for no alarm, so raising an alarm is preferable. This calculation assumes appropriate probabilities and the stated costs.

Alternatively, choose a threshold using validation observations to minimize $(\text{FP} + 5\text{FN})/m$. Sampling variation and imperfect probabilities can make it differ from $1/6$. Freeze the threshold before test evaluation. Do not refit the scoring model afterward without reconsidering validation: its probability scale may change.

One more regression metric. Application B also reports

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y}_{\text{eval}})^2}.$$

It can be negative on held-out rows and is undefined for constant evaluation targets. The evaluation mean occurs in this definition; it must not be used to fit the baseline predictor.

Remember. Select models with an appropriate loss. Select actions with a threshold and costs. Evaluate both on observations that did not select them.

6. The three companion notebooks

Each notebook puts statements before code and runs independently. Use seed 17 for simulation and seed 23 for the 60/20/20 split. Full tasks and outputs are in the notebooks.

A. Request latency, optimization, and collinearity

A_request_latency.ipynb

$$s \sim U(0.5, 10), \quad q \sim \text{Poisson}(3), \quad \epsilon \sim N(0, 12^2), \quad y = 40 + 8s + 3q + \epsilon,$$

with $n = 600$ independent requests and latency in ms. Fit mean, payload-only, and two-feature predictors. Compare the scalar formula, least squares, gradient descent, and a reference solver. Add $s' = s + N(0, 0.02^2)$; use resampling to contrast slope and prediction variability. Select ridge by training CV, inspect its path and residuals, and report test MAE/RMSE. **Main lesson:** a stable prediction does not require each correlated coefficient to be stable. Scaling aids optimization; ridge changes the objective.

B. Advertising, multiple features, and interactions

B_advertising.ipynb

Use the local 200-market `Advertising.csv`. The predictors are TV, radio, and newspaper budgets; the target is sales. Drop the CSV row index. Budgets and sales are in thousands of dollars and thousands of units.

Compare the mean baseline, TV-only, additive, and TV/radio/interaction candidates on identical training CV folds. Interpret $w_T + w_{TR}r$ at radio budgets 0 and 30. Inspect residuals; report frozen-model test MAE, RMSE, and R^2 against the baseline. **Main lesson:** feature construction can matter more than changing the optimizer. Select the relationship using held-out training folds.

C. Probability of a latency-budget violation

C_budget_violation.ipynb

$$z = -4 + 0.35s + 0.3q - 1.2c, \quad \pi = \sigma(z), \quad y \sim \text{Bernoulli}(\pi),$$

with $n = 2000$, $s \sim U(0.5, 10)$, $q \sim \text{Poisson}(3)$, and $c \sim \text{Bernoulli}(0.4)$ independent. Cache hit c is available at arrival. This is a separate generator, not a threshold of A's latency. Check the stable loss gradient and compare optimizers. Select an L_2 penalty by training CV log loss and a validation threshold for costs 1 and 5. Evaluate probability quality, calibration, precision-recall, and the prespecified test policies at 0.5, 1/6, and the selected threshold. **Main lesson:** accurate probabilities and a useful alarm policy are related but different objectives.

Further reading. The full first lecture is in PdfLectures. The bundled book is James et al., *An Introduction to Statistical Learning with Applications in R*, 2nd ed. (2021), Chapters 3, 4.3, and 6.2.1.